

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра дискретной математики и информационных технологий

**РАЗРАБОТКА ПРИЛОЖЕНИЯ ДЛЯ КЛАССИФИКАЦИИ
НОВОСТНЫХ СТАТЕЙ С ПОМОЩЬЮ BERT-МОДЕЛЕЙ**

БАКАЛАВРСКАЯ РАБОТА

студента 4 курса 421 группы
направления 09.03.01 — Информатика и вычислительная техника
факультета КНиИТ
Морозова Никиты Андреевича

Научный руководитель

к. э. н., доцент

Г. Ю. Чернышова

Заведующий кафедрой

к. ф.-м. н., доцент

Л. Б. Тяпаев

Саратов 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ методов решения задачи классификации текстов	5
1.1 Методы решения задач классификации текстов	5
1.2 Архитектура BERT-модели	6
2 Обучение BERT-моделей для классификации новостных сообщений	12
2.1 Определение гиперпараметров BERT-моделей	12
2.2 Способы анализа результатов классификации	16
2.3 Текстовый анализ новостных сообщений в соответствии с целями устойчивого развития	18
2.4 Выбор и адаптация моделей для классификации	27
3 Реализация интерфейса для классификации текстов	32
3.1 Разработка приложения для применения BERT-моделей	32
3.2 Пример применения приложения для классификации новостей	33
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38
Приложение А Программный код для предобработки набора данных.	41
Приложение Б Программный код для перевода текстов.	42
Приложение В Программный код для обучения моделей трансформеров ..	47

ВВЕДЕНИЕ

Классификация текстов является одной из ключевых задач обработки естественного языка (Natural Language Processing, NLP) и находит применение в таких областях, как анализ тональности отзывов, автоматическая модерация контента, категоризация документов, распознавание интенгов в чат-ботах и прочих. Традиционные методы, такие как алгоритмы на основе векторизации текстов и применении методов машинного обучения, часто сталкиваются с ограничениями в работе с контекстуальной неоднозначностью, синонимией и сложными семантическими связями. Это приводит к снижению качества предикативных моделей, особенно в сценариях, требующих глубокого понимания смысла текста.

С появлением сложных нейросетевых архитектур, в частности таких как трансформеры, произошел прорыв в NLP. Модели на основе BERT (Bidirectional Encoder Representations from Transformers) стали революционными благодаря своей способности учитывать двунаправленный контекст слов. В отличие от предшествующих подходов (например, LSTM), BERT обучается на предсказании пропущенных токенов и анализе связей между предложениями. Это позволяет модели формировать глубокие семантические представления текста, учитывая как локальные, так и глобальные зависимости.

В промышленных решениях BERT и его производные (например, RoBERTa, DistilBERT) успешно применяются для определения эмоциональной окраски текста (сентимент-анализ), автоматической категоризации новостных статей, фильтрации спама и токсичных сообщений, извлечения ключевой информации из юридических и медицинских документов.

Ключевым преимуществом BERT является возможность дообучения: модель, предобученная на больших корпусах текстов, может быть быстро дообучена на узкоспециализированных данных даже при ограниченном размере выборки. Это сокращает затраты на разметку данных и делает технологию доступной для малого бизнеса и научных проектов.

Рост объема цифровых текстовых данных и потребность в их автоматической обработке подчеркивают актуальность исследований в области применения BERT. В работе рассматриваются практические аспекты адаптации модели для конкретных сценариев классификации, а именно для классификации текстов по 17 целям устойчивого развития. Эта задача имеет ограниченное количество

обучающих выборок, что влияет на качество дообученных моделей.

Для русскоязычных текстовых документов эта задача широко не исследовалась, представляется актуальным исследовать возможность формирования обучающей выборки, обеспечивающей повышение точности классификационных моделей для русскоязычного контента в контексте целей устойчивого развития.

Целью данной бакалаврской работы является разработка приложения для классификации текстов в соответствии с целями устойчивого развития с помощью BERT-моделей. Для достижения поставленной цели требуется решить следующие задачи:

- отобрать и адаптировать наборы данных для обучения моделей;
- построить BERT-модели для классификации русскоязычных новостных сообщения по тематике, связанной с целями устойчивого развития;
- реализовать интерфейс для использования моделей.

Объектом работы является усовершенствование технологии классификации текстовых документов. Предметом работы являются методы глубокого обучения для решения прикладных задач.

Бакалаврская работа имеет следующую структуру. В первом разделе рассмотрены методы решения задач классификации текстов и трансформерные модели, в частности архитектура BERT. Во втором разделе описано обучение BERT-моделей для классификации новостных сообщений. В третьем разделе представлена реализация интерфейса для классификации текстов.

Работа содержит 9 рисунков, 3 таблицы и 30 источников.

1 Анализ методов решения задачи классификации текстов

1.1 Методы решения задач классификации текстов

Классификация текстов — задача автоматического присвоения категорий текстовым данным — прошла долгий путь развития, отражающий эволюцию технологий обработки естественного языка. Первые подходы к решению этой задачи основывались на ручном создании экспертных правил, таких как поиск ключевых слов или использование регулярных выражений. Например, позитивный отзыв мог определяться по наличию слов «отлично» или «рекомендую», а негативный — по словам «ужасно» или «разочарован». Однако такие системы были крайне трудоемкими в поддержке, плохо адаптировались к новым данным и не учитывали контекстную неоднозначность, например, иронию или сарказм.

С развитием машинного обучения фокус сместился на статистические методы. Тексты стали представляться в виде числовых векторов, таких как Bag of Words (BoW), где учитывалась частота слов, или TF-IDF, где вес слова зависел от его важности в документе и корпусе. Эти представления использовались в сочетании с алгоритмами вроде наивного Байеса, SVM (метод опорных векторов) или логистической регрессии. Такие методы позволяли автоматически выявлять закономерности в данных, например, классифицировать новости по темам (спорт, политика). Однако они игнорировали порядок слов и семантические связи, а высокая размерность данных (особенно для больших словарей) создавала проблему разреженности матриц.

Прорывом стало появление распределенных представлений слов (word embeddings), таких как Word2Vec, GloVe и FastText. Эти методы кодировали слова в плотные векторы, сохраняющие семантические и синтаксические связи — например, векторное представление слова «король» приближалось к разности векторов «королева» и «женщина». Это позволяло учитывать синонимию и контекстную близость слов, но у таких моделей оставался ключевой недостаток: статические эмбединги не могли отражать многозначность слов. Например, слово «ключ» в разных контекстах (инструмент, музыкальный знак, криптография) кодировалось одинаково [1].

Следующим этапом стали нейросетевые архитектуры. Сверточные сети (CNN) анализировали локальные n-граммы, извлекая признаки через фильтры, а рекуррентные сети (RNN и LSTM) обрабатывали тексты как последовательности, учитывая долгосрочные зависимости. Эти методы улучшили качество

классификации, особенно в задачах анализа тональности или категоризации длинных текстов, но столкнулись с проблемой «исчезающих градиентов» в RNN и однонаправленностью контекста — каждое слово зависело только от предыдущих токенов [2].

Радикальное изменение принесли трансформеры, представленные в 2017 году, и их потомок — модель BERT. Архитектура трансформеров, основанная на механизме самовнимания, позволяла анализировать зависимости между всеми словами в тексте одновременно, а двунаправленное обучение BERT учитывало как левый, так и правый контекст для каждого токена. Предобучение на огромных корпусах (например, Wikipedia) через задачи Masked Language Modeling (предсказание замаскированных слов) и Next Sentence Prediction (определение связи предложений) давало модели глубокое понимание семантики и синтаксиса. Это позволило дообучать BERT для узких задач классификации даже на небольших размеченных датасетах. Последующие модификации, такие как RoBERTa или DistilBERT, оптимизировали скорость и качество, сохраняя ключевые преимущества: способность работать с многозначностью, контекстной иронией и сложными семантическими связями. Также существуют модификации в виде многоязычных моделей, как Multilingual BERT и XLM-RoBERTa [3].

1.2 Архитектура BERT-модели

BERT (Bidirectional Encoder Representations from Transformers) [4] представляет собой фундаментальную нейросетевую архитектуру, разработанную Google в 2018 году. Её революционность заключается в использовании двунаправленного контекста при обработке текста, что принципиально отличает BERT от предыдущих подходов. Чтобы понять значение этого прорыва, необходимо рассмотреть исторические ограничения обработки последовательностей. До появления трансформеров доминировали рекуррентные нейронные сети (RNN), страдавшие двумя критическими недостатками. Во-первых, RNN хранят информацию о последовательности в скрытом состоянии, которое постоянно перезаписывается при обработке новых токенов. Это приводит к неизбежной потере контекста при работе с длинными текстами, поскольку модель физически не может сохранить все детали из начала последовательности к моменту достижения её конца. Во-вторых, последовательная природа вычислений в RNN делает их крайне неэффективными для параллелизации на GPU. Каждый шаг обработки зависит от результатов предыдущего, что создает вычислитель-

ное бутылочное горлышко при работе с объемными данными.

Решение этих проблем пришло с механизмом самовнимания (self-attention), который составляет ядро архитектуры трансформеров [5]. Этот подход позволяет каждому токenu в последовательности напрямую взаимодействовать с любым другим токеном, независимо от их относительного положения. Технически это реализуется через три обучаемых матрицы: запросов (Q), ключей (K) и значений (V). Для каждого токена вычисляются векторные представления: запрос определяет, какую информацию токен ищет, ключи служат индексами для поиска, а значения содержат собственно информацию. Веса внимания рассчитываются через функцию softmax от скалярных произведений запросов и ключей, нормированных на корень из размерности ключевого пространства. Формально это выражается как:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V,$$

где d_k — размерность векторов ключей. На рисунке 1 представлена визуализация устройства Multi-Head Attention.

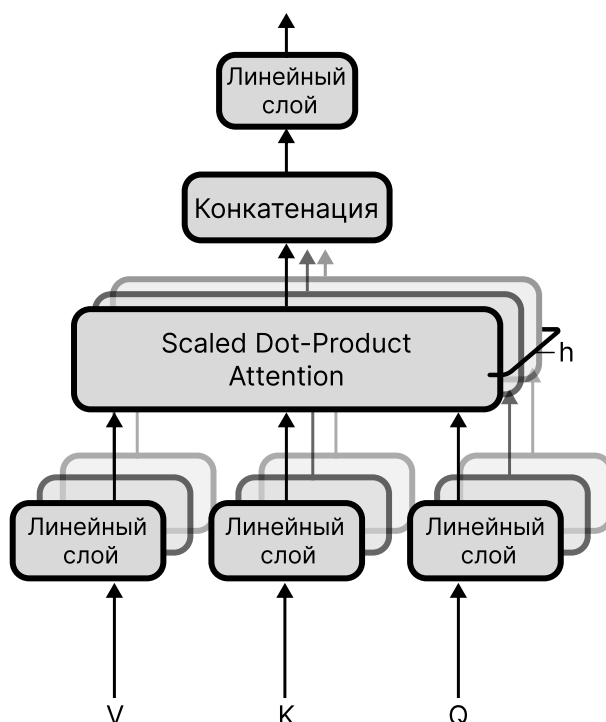


Рисунок 1 – Устройство Multi-Head Attention

Независимые модули внимания работают параллельно. Входные данные проецируются в разные пространства представлений через отдельные линейные

слои для Q, K, V. Выходы модулей конкатенируются и проецируются обратно в исходную размерность.

Преимуществом архитектуры является multi-head attention — механизм, при котором несколько модулей внимания работают параллельно. Каждый модуль изучает различные типы зависимостей в тексте (например, синтаксические и семантические), а их выходы объединяются через конкатенацию и линейную проекцию. Это позволяет модели одновременно улавливать разнородные языковые закономерности, что принципиально невозможно в однонаправленных архитектурах [6].

BERT использует исключительно кодирующую часть трансформера (encoder), состоящую из стекированных идентичных слоев. Каждый слой содержит два ключевых компонента: блок multi-head attention и полносвязную прямую сеть (feed-forward network). После каждой операции применяются остаточные связи (residual connections) и нормализация слоя (layer normalization), обозначенные на рисунке 2 как Add & Norm.

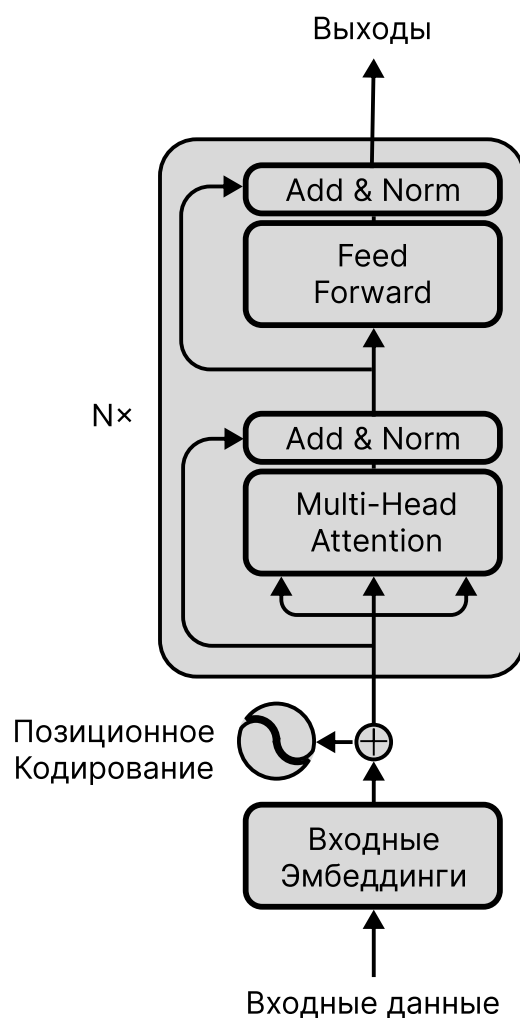


Рисунок 2 – Архитектура кодировщика

Кодировщик трансформера состоит из N идентичных слоев. Каждый слой содержит блок Multi-Head Attention, Feed-Forward Network, остаточные связи (Add) и нормализацию (Norm). Стрелки показывают поток данных через эти компоненты.

Эта конструкция обеспечивает стабильность обучения глубоких сетей. Важнейшее отличие BERT от других архитектур — его двунаправленность. В отличие от моделей, где контекст накапливается только слева направо, BERT анализирует полное окружение каждого слова, одновременно учитывая как предшествующий, так и последующий контекст. Это особенно критично для разрешения лексической многозначности, когда значение слова определяется всем предложением.

Предобучение BERT осуществляется на двух специальных задачах. Первая — Masked Language Modeling (MLM), где случайные токены в тексте заменяются маской [MASK], а модель обучается их восстанавливать по контексту.

Вторая — Next Sentence Prediction (NSP), где модель определяет, следует ли одно предложение за другим в оригинальном тексте. Этот этап позволяет BERT освоить как лексико-семантические, так и дискурсивные закономерности языка.

Поскольку механизм внимания инвариантен к порядку элементов, для учета позиционной информации используются позиционные эмбединги. В оригинальной реализации применяются фиксированные синусоидальные кодировки, вычисляемые по формулам:

$$E_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d}} \right),$$

$$E_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d}} \right),$$

где pos — позиция токена, i — индекс в векторе эмбединга, d — размерность эмбединга. Эти векторы добавляются к токеным эмбедингам перед подачей на вход кодировщика.

Для задач классификации используется выходное представление специального токена [CLS], который добавляется в начало последовательности. Считается, что этот вектор агрегирует информацию о всем документе. К нему добавляется классификатор — обычно один или несколько полносвязных слоев, — который дообучается на целевом наборе данных. На рисунке 3 показано, как входное предложение преобразуется через комбинацию токенных, сегментных и позиционных эмбедингов.



Рисунок 3 – Графический пример работы BERT с предложениями

Входные токены преобразуются через три типа эмбедингов: токенные (Token Embeddings), сегментные (Segment Embeddings) для различения пар

предложений, и позиционные (Position Embeddings). Их суммарный вектор служит входом для кодировщика.

Обучение таких моделей требует специальных подходов. Большие батчи (часто эквивалентные тысячам токенов) и оптимизаторы типа AdamW помогают стабилизировать процесс. На рисунке 3 демонстрируется отличие механизма внимания в BERT от генеративных моделей: в BERT отсутствует маскирование будущих токенов, что обеспечивает полный доступ к контексту. Архитектурные особенности делают BERT особенно эффективным для задач понимания текста, где требуется глубокое контекстуальное моделирование, что подтверждается его превосходством в таких задачах, как классификация новостных статей по целям устойчивого развития.

Далее в работе будут использоваться различные модификации BERT-моделей, как наиболее перспективный подход в решении задачи классификации текстов.

2 Обучение BERT-моделей для классификации новостных сообщений

2.1 Определение гиперпараметров BERT-моделей

Для задачи классификации текстов при помощи BERT-модели существует подход к обучению путём добавления слоёв к BERT-модели с целью конвертации трёхмерной матрицы, получаемой на выходе BERT и дальнейшего получения предсказания метки класса, гиперпараметрами при таком подходе являются [7]:

- количество дополнительных слоёв;
- функция потерь;
- оптимизатор.

Количество скрытых слоёв является гиперпараметром в выбранном подходе к обучению BERT-модели для задачи классификации. У каждого слоя нейронной сети есть своя функция активации. Она определяет то, в каком диапазоне будут выходные значения нейронов этого слоя. Рассмотрим список возможных функций активации [8].

Линейная функция активации: это самая простая функция активации. Она просто возвращает входное значение без изменений:

$$f(x) = x.$$

Ступенчатая функция активации (функция Хевисайда): эта функция возвращает 0, если входное значение меньше некоторого порога, и 1, если входное значение больше или равно порогу:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}.$$

Сигмоидная функция активации: эта функция возвращает значение между 0 и 1. Она часто используется в выходном слое нейронных сетей для задач бинарной классификации:

$$f(x) = \frac{1}{1 + e^{-x}}.$$

Гиперболический тангенс (tanh): эта функция возвращает значение между

-1 и 1. Она похожа на сигмоидную функцию, но может обрабатывать отрицательные входные значения:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

ReLU (Rectified Linear Unit): эта функция возвращает входное значение, если оно положительное, и 0, если оно отрицательное. Она широко используется в скрытых слоях нейронных сетей:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}.$$

GeLU сочетает свойства ReLU и гауссова распределения, плавно взвешивая входные значения на основе их вероятности. Широко используется в трансформерных архитектурах:

$$f(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left(1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right),$$

где $\Phi(x)$ — кумулятивная функция распределения стандартной нормальной величины,

$$\operatorname{erf}(x) \text{ — функция ошибок: } \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt.$$

Softmax: эта функция преобразует числовые значения в вероятности. Она используется в выходном слое нейронных сетей для задач многоклассовой классификации. Формула для i -го элемента вектора x :

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}},$$

где N - это количество элементов в векторе x .

Функция потерь — в большинстве обучающих сетей ошибка рассчитывается как разница между фактическим выходным значением и прогнозируемым выходным значением. Функция, используемая для вычисления этой ошибки, известна как функция потерь, также часто называемая функцией ошибки или затрат [9]. Существует большое количество функций потерь, но в рамках классификации будут рассмотрены только некоторые, которые доступны для данной задачи на платформе Python.

Кросс-энтропия (логарифмическая функция потерь, $\log \text{ loss}$). Это стандартная функция потерь для многоклассовой классификации. Она измеряет разницу между истинными и предсказанными вероятностями классов. Если кросс-энтропия велика, это означает, что разница между двумя распределениями велика, а если кросс-энтропия мала, то распределения похожи друг на друга. Формула кросс-энтропии выглядит так:

$$L = - \sum_{c=1}^N y_c \log(p_c),$$

где y_c — это истинная метка класса, p_c — это предсказанная вероятность для этого класса.

Фокусная кросс-энтропия — это модифицированная версия кросс-энтропии, которая вводит концепцию фокуса. Она увеличивает потери для тех примеров, которые модель предсказывает неверно, и уменьшает потери для тех примеров, которые модель предсказывает верно. Это делает ее полезной в сценариях, где присутствует дисбаланс классов. Она была предложена в работе “Focal Loss for Dense Object Detection” для решения проблемы дисбаланса классов в задачах детекции объектов. Focal Loss определяется следующим образом [10]:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t),$$

где p_t — вероятность принадлежности к целевому классу, предсказанная моделью, α_t — весовой коэффициент, который используется для управления вкладом каждого класса в общую функцию потерь, γ — фокусный параметр, который регулирует степень увеличения вклада трудноклассифицируемых примеров. Большие значения γ увеличивают вклад трудноклассифицируемых примеров.

Оптимизатор в нейронной сети — это алгоритм или метод, который используется для изменения атрибутов нейронной сети, таких как веса и скорость обучения, с целью уменьшения потерь.

Оптимизаторы играют ключевую роль в обучении нейронных сетей. Они определяют, как обновляются веса сети на основе градиента функции потерь. Оптимизаторы также могут помочь в предотвращении проблем, таких как застревание в локальных минимумах или проблемы с затухающими или взрывающимися градиентами.

Существует множество различных типов оптимизаторов, включая сто-

хастический градиентный спуск (SGD), Adam, RMSProp и многие другие. В нейронных сетях для задач классификации обычно используются следующие оптимизаторы [11].

Стохастический градиентный спуск (SGD) — это самый базовый оптимизатор. Он обновляет веса модели, делая шаг в направлении, обратном градиенту функции потерь.

$$w_{t+1} = w_t - \eta \nabla J(w_t),$$

где w_t — это веса на шаге t , η — это скорость обучения, $\nabla J(w_t)$ — это градиент функции потерь J по отношению к весам на шаге t .

Моментум (Momentum) — это вариант SGD, который учитывает предыдущие шаги для сглаживания обновлений весов. Это помогает ускорить SGD в правильном направлении и смягчить колебания.

$$v_{t+1} = \mu v_t - \eta \nabla J(w_t),$$

$$w_{t+1} = w_t + v_{t+1},$$

где v_t — скорость на шаге t , μ — это коэффициент момента.

Adagrad — этот оптимизатор адаптирует скорость обучения для каждого параметра, делая большие обновления для редко встречающихся и малые обновления для часто встречающихся параметров.

$$G_{t+1} = G_t + (\nabla J(w_t))^2,$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_{t+1} + \epsilon}} \nabla J(w_t),$$

где G_t — диагональная матрица, которая содержит сумму квадратов градиентов до шага t , ϵ — это небольшое число для предотвращения деления на ноль.

RMSprop также адаптирует скорость обучения для каждого параметра и очень эффективен в случаях с очень шумными или автокоррелированными градиентами.

$$E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2,$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t,$$

где $E[g^2]_t$ — оценка второго момента (ненормированной дисперсии) градиента.

Adam (Adaptive Moment Estimation) сочетает в себе идеи Momentum и RMSprop. Он сохраняет экспоненциально убывающее среднее прошлых градиентов, подобно Momentum, и экспоненциально убывающее среднее прошлых квадратов градиентов, подобно RMSprop.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t,$$

где m_t и v_t — оценки первого момента (среднего значения) и второго момента градиента соответственно.

AdamW — это вариант Adam, который имеет более правильное взаимодействие с регуляризацией весов. Формула для AdamW очень похожа на формулу для Adam, но с добавлением штрафа L2 для весов:

$$w_{t+1} = (1 - \eta\lambda)w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t,$$

где λ — коэффициент регуляризации L2.

Выбор оптимизатора зависит от конкретной задачи и данных. Всегда стоит провести серию экспериментов, чтобы определить, какой оптимизатор работает лучше всего для конкретной задачи.

2.2 Способы анализа результатов классификации

В рамках анализа результатов классификации используют следующие понятия:

- TP — число объектов, которые классификатор верно отнёс объект к рассматриваемому классу;
- TN — число объектов, которые классификатор верно утверждает, что объект не принадлежит к рассматриваемому классу;
- FP — число объектов, которые классификатор неверно отнёс объект к

рассматриваемому классу;

- FN — число объектов, которые классификатор неверно утверждает, что объект не принадлежит к рассматриваемому классу.

В качестве оценок существуют следующие метрики [12] (для бинарной классификации).

Accuracy (точность) показывает долю правильных классификаций. Несмотря на очевидность и простоту, является одной из самых малоинформативных оценок классификаторов:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Recall (полнота, sensitivity, TPR (true positive rate)) показывает отношение верно классифицированных объектов класса к общему числу элементов этого класса:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Precision (точность, перевод совпадает с accuracy) показывает долю верно классифицированных объектов среди всех объектов, которые к этому классу отнес классификатор:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

F_1 -мера среднее гармоническое между precision и recall. Формула:

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ввиду того, что такие оценки никак не учитывают изначальное распределение классов в выборке (что может существенно влиять на полученное значение), также существуют взвешенные варианты этих оценок (в терминах многоклассовой классификации).

Формула для вычисления Accuracy для многоклассовой классификации:

$$\text{Accuracy} = \frac{\sum_{i=1}^K \text{Accuracy}_i}{K},$$

где K - количество классов,

i - номер класса.

Формула для вычисления Recall для многоклассовой классификации:

$$\text{Recall} = \frac{\sum_{i=1}^K \text{Recall}_i}{K},$$

где K - количество классов,

i - номер класса.

Формула для вычисления Precision для многоклассовой классификации:

$$\text{Recall} = \frac{\sum_{i=1}^K \text{Recall}_i}{K},$$

где K - количество классов,

i - номер класса.

F_1 -мера — среднее гармоническое между precision и recall:

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

С помощью этих метрик, можно оценивать качество моделей машинного обучения. В данной работе все перечисленные метрики будут использоваться комплексно, что позволит получить всестороннюю оценку качества модели и обеспечить надежность результатов классификации.

2.3 Текстовый анализ новостных сообщений в соответствии с целями устойчивого развития

Цели устойчивого развития (ЦУР, Sustainable Development Goals, SDG) и необходимых для их достижения 169 задач (targets) принятые Организацией Объединенных Наций в 2015 году, представляют собой амбициозный план из 17 взаимосвязанных задач, направленных на преобразование мира к 2030 году. Эти цели охватывают широкий спектр глобальных проблем: от ликвидации нищеты (ЦУР 1) и голода (ЦУР 2) до обеспечения доступа к чистой энергии (ЦУР 7), борьбы с изменением климата (ЦУР 13) и сохранения морских экосистем (ЦУР 14). Полный список включает:

1. повсеместная ликвидация нищеты во всех ее формах;
2. ликвидация голода, обеспечение продовольственной безопасности и улучшение питания и содействие устойчивому развитию сельского хозяйства;
3. обеспечение здорового образа жизни и содействие благополучию для всех

- в любом возрасте;
4. обеспечение всеохватного и справедливого качественного образования и поощрение возможности обучения на протяжении всей жизни для всех;
 5. обеспечение гендерного равенства и расширение прав и возможностей всех женщин и девочек;
 6. обеспечение наличия и рационального использования водных ресурсов и санитарии для всех;
 7. обеспечение всеобщего доступа к недорогим, надежным, устойчивым и современным источникам энергии для всех;
 8. содействие поступательному, всеохватному и устойчивому экономическому росту, полной и производительной занятости и достойной работе для всех;
 9. создание стойкой инфраструктуры, содействие всеохватной и устойчивой индустриализации и инновациям;
 10. сокращение неравенства внутри стран и между ними;
 11. обеспечение открытости, безопасности, жизнестойкости и экологической устойчивости городов и населенных пунктов;
 12. обеспечение перехода к рациональным моделям потребления и производства;
 13. принятие срочных мер по борьбе с изменением климата и его последствиями;
 14. сохранение и рациональное использование океанов, морей и морских ресурсов в интересах устойчивого развития;
 15. защита и восстановление экосистем суши и содействие их рациональному использованию, рациональное лесопользование, борьба с опустыниванием, прекращение и обращение вспять процесса деградации земель и прекращение процесса утраты биоразнообразия;
 16. содействие построению миролюбивого и открытого общества в интересах устойчивого развития, обеспечение доступа к правосудию для всех и создание эффективных, подотчетных и основанных на широком участии учреждений на всех уровнях;
 17. укрепление средств осуществления и активизация работы в рамках Глобального партнерства в интересах устойчивого развития [13].

Каждая цель сопровождается конкретными индикаторами, которые поз-

воляют оценивать прогресс на национальном и международном уровнях [14].

Цели охватывают три ключевых измерения устойчивого развития:

- экономическое развитие — ликвидация бедности (Цель 1), обеспечение достойной работы (Цель 8), инновации (Цель 9);
- социальное развитие — качественное образование (Цель 4), гендерное равенство (Цель 5), здоровье (Цель 3);
- экологическая устойчивость — борьба с изменением климата (Цель 13), сохранение экосистем суши (Цель 15) и океанов (Цель 14).

Россия активно участвовала в формировании ЦУР, была одной из 193 стран, которые утвердили ЦУР на Генеральной Ассамблее ООН в сентябре 2015 г. Представители РФ участвуют в работе тематических групп. Россия включила задачи ЦУР в свои национальные проекты: из 169 задач Повестки-2030 российские национальные проекты напрямую или косвенно охватывают 107. В июле 2020 г. Россия представила Добровольный национальный обзор (ДНО) о прогрессе в достижении ЦУР на Политическом форуме ООН. Доклад был подготовлен аналитическим центром при Правительстве РФ совместно с Минэкономразвития, МИД России, Росстатом и другими ведомствами. На форуме был представлен также независимый гражданский обзор ЦУР «2020–2030: Десятилетие действий для ЦУР в России. Вызовы и решения». Отслеживание прогресса в научных исследованиях в контексте ЦУР является важной задачей мониторинга устойчивого развития и само является предметом многих исследований [15].

Автоматизация анализа таких текстов становится критически важной в условиях стремительного роста медиаданных. По оценкам, ежедневно в мире публикуется свыше 3 миллионов новостных статей [16], и их ручная обработка непрактична. Традиционные методы классификации, основанные на ключевых словах (например, поиск термина «изменение климата» для ЦУР 13), часто терпят неудачу из-за многозначности языка. Фраза «зелёный рост» может относиться как к экологическим технологиям (ЦУР 7, 9), так и к финансовым показателям компании, а ироничный заголовок «Отличный способ борьбы с бедностью!» в контексте коррупционного скандала и вовсе вводит алгоритмы в заблуждение.

В задачах обработки текстовой информации применяются как традиционные методы обработки естественного языка (NLP), так и модели глубокого

обучения. Однако классические алгоритмы, такие как SVM (метод опорных векторов) и наивный Байес, хотя и показывали высокие результаты в бинарной классификации, однако уступали трансформерам в многоклассовых задачах из-за ограничений в учете контекста.

Современные исследования в области автоматической обработки текстовых данных для определения их соответствия целям устойчивого развития (ЦУР) направлены на решение задач классификации, выявления взаимосвязей между целями и повышения эффективности анализа больших массивов документов. Сообщество стремится разработать методы, способные автоматически определять релевантность текстовых фрагментов конкретным ЦУР, что особенно актуально в контексте роста числа корпоративных отчетов, политических документов и научных публикаций, связанных с устойчивым развитием.

Один из подходов к решению задачи классификации текстов, связанных с ЦУР, представлен в исследовании [17]. Авторы предложили метод, основанный на модели BERT, для автоматического определения принадлежности текстов сразу к нескольким ЦУР. Ключевым этапом стало создание размеченного датасета, который включал 6017 текстов, охватывающих все 17 целей. Первоначальный набор данных был сформирован на основе описаний ЦУР из официальных документов ООН, дополненных сгенерированными текстами с помощью библиотеки Markovify и определениями синонимов, извлеченными через API Wikipedia.

Тексты были преобразованы в формат, совместимый с BERT, включая токенизацию и приведение к нижнему регистру. К предобученной модели BERT был добавлен слой для многометочной классификации, что позволило оценивать вероятность принадлежности текста к каждой из 17 ЦУР. Использовались параметры, оптимальные для задач NLP: оптимизатор AdamW с learning rate = $1e-5$; Dropout для предотвращения переобучения.

Исследование столкнулось с ограниченностью исходных данных. Начальный датасет, построенный на текстах ООН, имел узкий тематический диапазон, что потребовало дополнения с помощью синтетических примеров. Датасет был создан вручную авторами исследования. Чтобы увеличить объем и разнообразить стили текстов, авторы выделили список наиболее частых слов для каждой из целей и использовали инструмент Markovify для генерации искусственных текстов, было сгенерировано по 300 новых текстов на каждую из 17 целей, то

есть всего 5100 синтетических текстов. Изначально их было 917.

Результаты исследования показали высокую точность модели — $F1 = 0.94$, что подтверждает эффективность BERT для задач многометочной классификации. Тестирование на реальных текстах из платформ SDG Pathfinder и IIDSD продемонстрировало способность модели корректно идентифицировать основные и сопутствующие цели.

Ключевым ресурсом для исследования [18] стал OSDG Community Dataset (OSDG-CD), содержащий тексты из публичных отчетов, политических документов и научных статей, аннотированные более чем 1000 волонтеров. Датасет охватывает 15 из 17 ЦУР (SDG1–SDG15), при этом средняя длина текстовых фрагментов составляет около 90 токенов. Для повышения качества данных исследователи применяли строгую фильтрацию: удаляли записи с низким уровнем согласия между аннотаторами ($\text{agreement} < 0.6$).

При обучении моделей BERT и RoBERTa использовались стандартные гиперпараметры: 5 эпох, оптимизатор AdamW с $\text{learning rate} = 1e-5$ и размером батча 3. Токенизация выполнялась с ограничением длины последовательностей до 512 токенов, что соответствует возможностям большинства трансформерных архитектур. Для классических методов, таких как SVM, применялось линейное ядро с параметром регуляризации $C=1.0$, а логистическая регрессия настраивалась с $\text{penalty}='l2'$ и $\text{solver}='lbfgs'$.

Наибольшую эффективность продемонстрировали модели на основе RoBERTa. В многоклассовой классификации их F1-мера была — 0.92, что на 1–2% превосходило результаты BERT. Традиционные методы, такие как SVM и логистическая регрессия, показывали сопоставимый результат в бинарных сценариях, но их производительность снижалась при работе с редкими классами (например, SDG10 «Уменьшение неравенства» и SDG12 «Ответственное потребление»). Среди ключевых проблем исследований выделялись:

- несбалансированность данных — некоторые ЦУР были представлены недостаточным количеством примеров;
- отсутствие 16 и 17 целей в наборе данных;
- субъективность аннотаций — даже после фильтрации оставался риск шума в данных из-за расхождений в оценках волонтеров.

Проведенный анализ подтверждает, что современные трансформерные архитектуры, в частности BERT, являются наиболее перспективным инструментом

для автоматизации обработки отчетов по ЦУР. Их способность учитывать контекст и семантические связи между словами обеспечивает превосходство над классическими методами. Однако для достижения максимальной точности требуется дальнейшая работа по улучшению датасетов — устранению дисбалансов и стандартизации аннотаций.

Новостные сообщения играют ключевую роль в мониторинге ЦУР, так как они оперативно отражают события, инициативы и вызовы, связанные с устойчивым развитием. Например, статья о внедрении инклюзивного образования может косвенно указывать на прогресс в ЦУР 4 («Качественное образование»), а репортаж о стихийном бедствии — актуализировать ЦУР 13 («Борьба с изменением климата»). Однако тексты СМИ редко структурированы в соответствии с официальными индикаторами ООН. Они часто содержат неявные упоминания целей, контекстные метафоры или пересекающиеся темы, такие как связь между «зелёными инвестициями» (ЦУР 9 и 13) или гендерным равенством в STEM-профессиях (ЦУР 5 и 8).

Этот вызов требует применения моделей, способных понимать семантику и контекст. BERT (Bidirectional Encoder Representations from Transformers), с его способностью анализировать двунаправленные зависимости между словами, становится мощным инструментом для решения подобных задач. Например, в статье о запуске солнечной электростанции в сельской местности модель может распознать связь между терминами «доступность энергии», «сокращение выбросов» и ЦУР 7, даже если последняя явно не упоминается. В данной работе BERT-модели адаптируются для классификации новостей по всем 17 ЦУР, учитывая их взаимосвязи и контекстуальные нюансы.

Процесс формирования наборов данных для обучения модели BERT основывался на строгих требованиях к объему, качеству и релевантности текстов. Первостепенное внимание уделялось обеспечению достаточного размера выборок — в исследование включались только коллекции, содержащие более 4000 текстов, так как меньший объем данных не позволяет эффективно дообучать модель с учетом специфики ЦУР. Кроме того, критическим критерием стала минимальная длина текста: статьи короче 512 символов, такие как краткие анонсы или заголовки без детализации, удалялись из-за недостаточной информативности для анализа контекста.

Особую сложность представляла сбалансированность классов. Наборы

данных с выраженным доминированием отдельных ЦУР (например, 80% статей, связанных с изменением климата) либо исключались, либо корректировались методами аугментации. Качество разметки также играло ключевую роль: приоритет отдавался наборам с ручной аннотацией экспертами, тогда как автоматически размеченные данные (например, через ключевые слова) отсеивались.

Важным ограничением исследования стало отсутствие русскоязычных наборов данных, соответствующих критериям ЦУР. Все отобранные коллекции, включая, представлены на английском языке, что обусловлено доминированием англоязычных источников в открытых репозиториях, посвященных устойчивому развитию.

Список всех наборов данных, их характеристики и результат отбора представлены в таблице 1.

Таблица 1 – Характеристики отобранных наборов данных

Набор данных	Объем	Примечание
EUR-Lex SDG-Annotated Dataset [20]	57000	Исключен (дисбаланс классов)
UNO Sustainable Development Goals [21]	4613	Исключен (короткие тексты, дубликаты)
OSDG Community Dataset (OSDG-CD) [22]	43025	Включен
SDG-single [23]	14219	Исключен (дисбаланс, короткие тексты)
SDGi Corpus [24]	7350	Включен
OSDG SDG Dataset [25]	5300	Исключен (только 14 из 17 классов)
SDGdescriptions-AURORA05v2 [26]	76959	Исключен (короткие тексты)

Основой исследования стали два специализированных корпуса текстовых данных. OSDG Community Dataset (OSDG-CD) объемом 43 025 текстов был сформирован в результате краудсорсинговой инициативы OSDG Community Platform с участием более 1000 волонтеров из 100 стран. Каждый текст представляет собой фрагмент публичных документов, таких как отчеты, политические заявления и научные публикации. Для обеспечения надежности данных использовалась многоэтапная валидация, включающая минимум три независимых оценщика. Они определяли соответствие текста одной из 16 ЦУР через бинарное голосование. В процессе работы из набора данных были исключены

тексты которые не обладали достаточной длиной и те, для которых согласие экспертов было меньше 70% и добавлены тексты, относящиеся к 17 цели из набора данных SDGi Corpus, так как в исходном наборе они отсутствовали, таким образом осталось 21355 семплов.

Распределение данных по классам для этого набора продемонстрировано на рисунке 4.

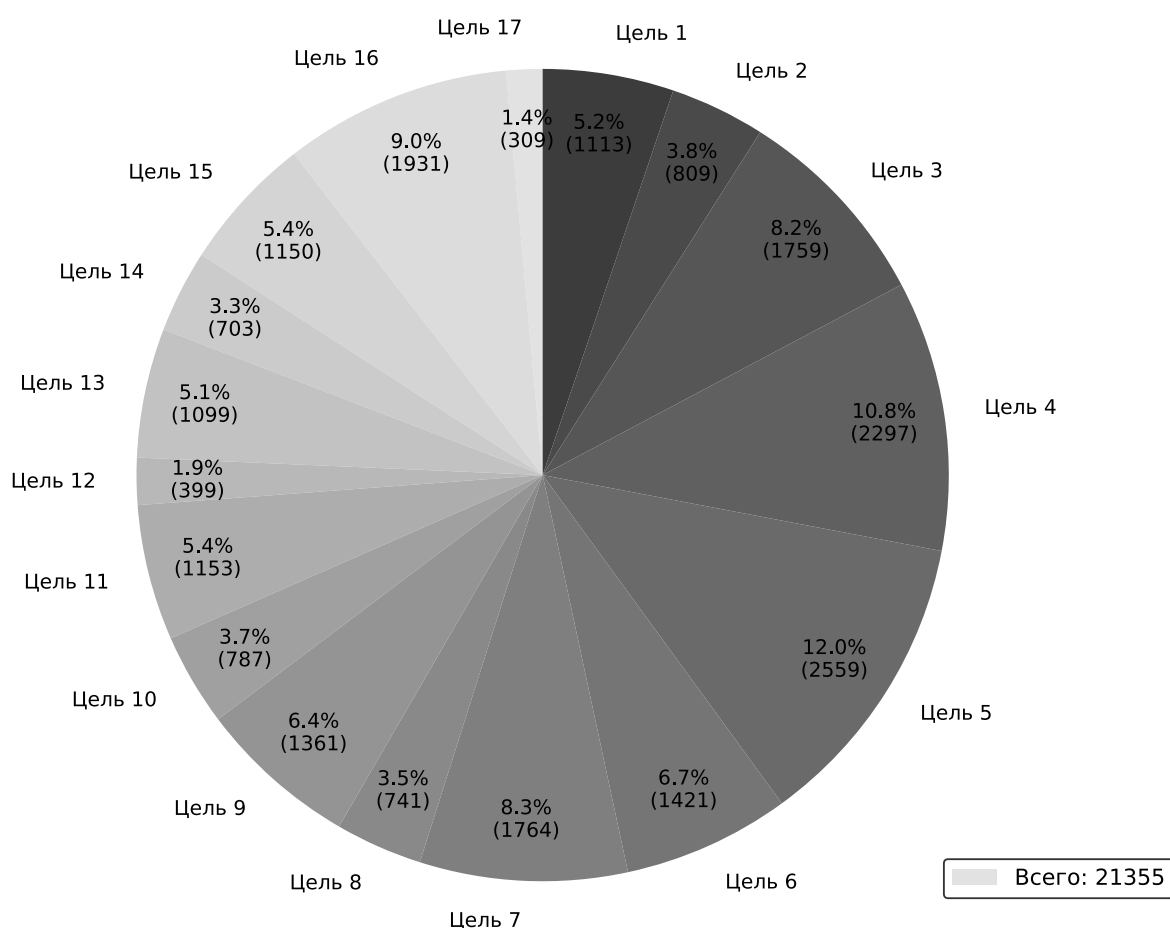


Рисунок 4 – Распределение данных по классам для OSDG-CD

Вторым источником данных является SDGi Corpus, включающий 7 350 текстов. Это многоязычная коллекция официальных документов ООН на английском, французском и испанском языках, содержащая Добровольные национальные и местные обзоры за 2016–2023 годы. Отличительной чертой датасета является строгая привязка к первоисточникам: метки ЦУР заимствованы непосредственно из структуры отчетов, что гарантирует высокую достоверность данных. В процессе работы из набора данных были исключены тексты которые не обладали достаточной длиной, таким образом осталось 4405 семплов. Распределение выборки по целям устойчивого развития для набора SDGi Corpus

продемонстрировано на рисунке 5.

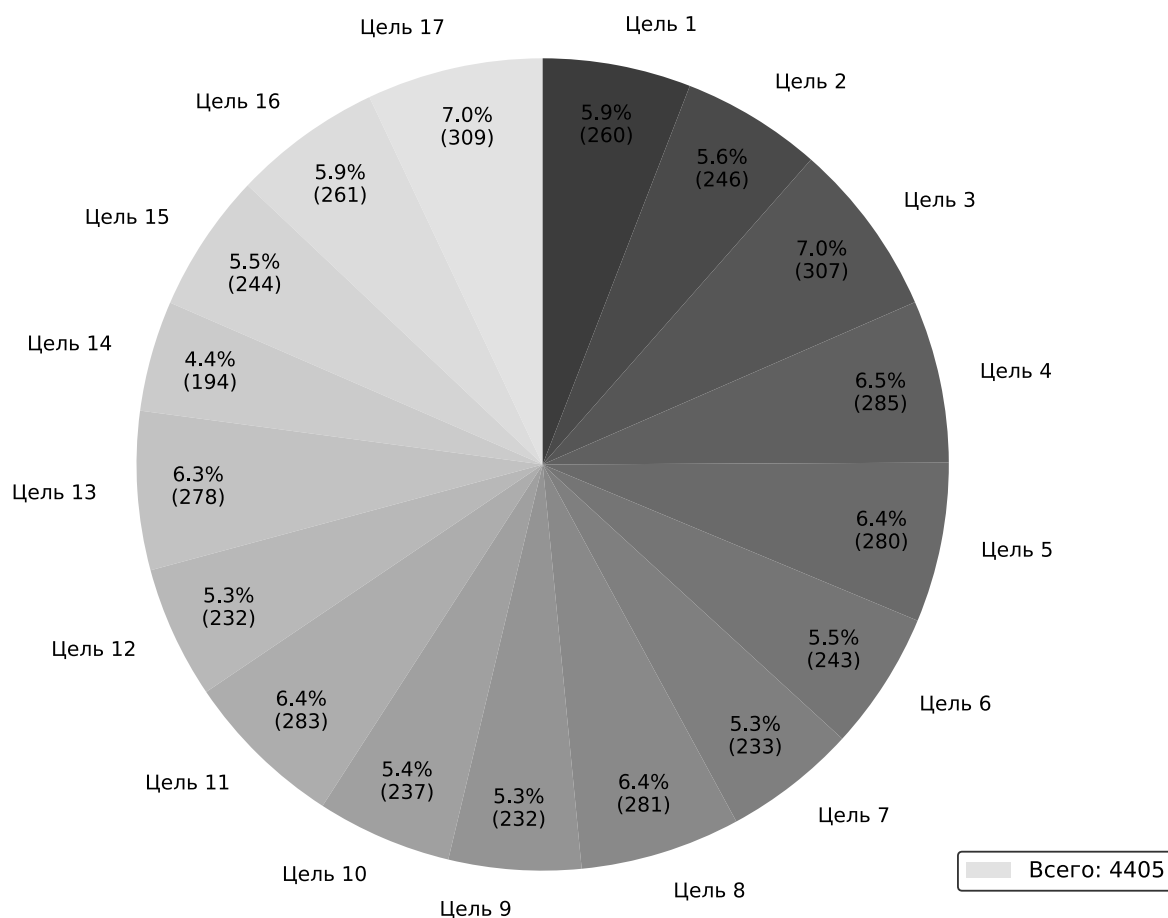


Рисунок 5 – Распределение выборки по ЦУР для SDGi Corpus

Объединенный корпус данных объемом 50 375 текстов был подвергнут предобработке. На первом этапе удалялись слишком короткие тексты, содержащие менее 256 символов, поскольку они могли не иметь достаточного контекста. Затем проводилась очистка от технических артефактов, включая специальные символы, HTML-теги и цифровые последовательности. Далее тексты нормализовались путем приведения к нижнему регистру, удаления пунктуации и стоп-слов с использованием библиотеки NLTK, дополненной терминами вне контекста ЦУР. Для совместимости с архитектурой BERT была проведена векторизация меток посредством LabelEncoder [19]. После предобработки осталось 26910 семплов. Детали реализации описаны в Приложении А.

Для интеграции англоязычных источников в русскоязычную модель классификации был разработан конвейер автоматизированного перевода и очистки текстов. Основу процесса составил пакетный перевод через API Google Translate с использованием библиотеки ‘deep_translator’, оптимизированный параллель-

ной обработкой данных средствами ‘ThreadPoolExecutor’.

Было реализовано кэширование результатов перевода. Система сохраняла переводы в CSV-файл и загружала их при повторной обработке. Тексты разбивались на пакеты по 50 элементов и переводились.

Для обеспечения отказоустойчивости алгоритм автоматически повторял запросы после 10-секундной паузы при возникновении ошибок подключения.

В результате обработки корпуса SDGi Corpus получено 5.2 млн токенов после очистки. Средняя скорость перевода составила 120 текстов в минуту.

Полный код, включая механизмы кэширования, обработки исключений и параллельного выполнения, представлен в Приложении Б. Разработанная методология обеспечила совместимость иноязычных документов с русскоязычной NLP-моделью без потери смысловой связи с индикаторами ЦУР.

2.4 Выбор и адаптация моделей для классификации

Для классификации текстов на разноязычных данных выбраны следующие предобученные модели: мультиязычные BERT-base-multilingual-cased [27] и XLM-RoBERTa-base [29], а также русскоязычная RuBERT-base-cased [28]. Первые две модели поддерживают обработку текстов на 100+ языках, включая русский и английский, что позволяет работать с гетерогенными данными без разделения на языковые подмножества, тогда как RuBERT специализирована исключительно для русского языка.

BERT-base-multilingual-cased (178 млн параметров) предобучена на корпусах Wikipedia для 104 языков с использованием токенизации WordPiece, что обеспечивает устойчивость к морфологической сложности русского языка.

RuBERT-base-cased (178 млн параметров) представляет собой русскоязычную специализацию BERT, дообученную на обширных русскоязычных корпусах, включая новостные тексты, Wikipedia и данные социальных медиа, что значительно улучшает качество обработки русскоязычных текстов.

XLM-RoBERTa-base (270 млн параметров), обученная на 2.5 ТБ данных CommonCrawl, демонстрирует повышенную эффективность при анализе длинных контекстов благодаря методу маскирования целых слов (Whole Word Masking), что особенно актуально для многоабзачных отчетов ООН.

Проблемой стало отсутствие специализированных моделей, предобученных на задачах классификации по ЦУР. Это потребовало адаптации выбранных архитектур через дообучение на размеченных данных с учетом специфики

устойчивого развития. Наличие RuBERT в данном наборе особенно ценно, так как обеспечивает глубокую лингвистическую адаптацию для русскоязычных документов.

В рамках отдельных вычислительных экспериментов каждая из этих моделей тестировалась на предмет изменения основных гиперпараметров, однако это не привело к значительному увеличению точности. Далее в работе представлены модели с наборами характеристик из таблицы 2.

Таблица 2 – Модифицированные модели

Модель	Исходная модель	Доп. слои	Оптимизатор	Функция потерь
M1	Multilingual BERT	256 ReLU Dropout(30%)	Adam	cross-entropy
M2	RuBERT	256 ReLU Dropout(30%)	Adam	cross-entropy
M3	XLM-RoBERTa	256 ReLU Dropout(30%)	Adam	cross-entropy

Модели адаптированы для задачи классификации через добавление двух-слойного классификатора поверх выходных эмбеддингов [CLS]:

- промежуточный Dense-слой (256 нейронов, ReLU) для выявления кросс-языковых семантических паттернов ЦУР;
- финальный Dense-слой с 17 нейронами и softmax-активацией, соответствующий числу ЦУР.

Для компенсации дисбаланса классов применены сбалансированные веса, рассчитанные через функцию ‘compute_class_weight’ библиотеки scikit-learn. Это позволило повысить F1-меру для малочисленных классов.

Обучение проводилось с использованием оптимизатора Adam (learning rate=2e-5, batch size=16). Для предотвращения переобучения задействованы:

- dropout (30%) в классификаторе;
- ранняя остановка (patience=5 эпох) по метрике F1;
- динамическое снижение learning rate при плато качества (factor=0.1).

Модели показали сопоставимые результаты на тестовой выборке, подтвердив применимость трансформеров для мультиязычной классификации в контексте устойчивого развития. Полная их реализация находится в Приложении В.

Для объективной оценки моделей сформирована комбинированная те-

стовая выборка, репрезентативно покрывающая все классы ЦУР. В неё вошли данные из трёх источников: 1459 текстов из объединённого датасета SDGi и OSDG, 1091 текст из SDG Knowledge Hub (международные документы ООН). Все данные переведены на русский язык. Выборка построена методом стратифицированного семплирования с балансировкой по классам: для каждой из 17 ЦУР отобрано ровно 150 текстов, что в сумме составляет 2550 примеров. Это позволило исключить перекося в сторону многообъектных классов. Результаты тестирования обученных моделей на различных наборах данных в виде значений метрик предоставлены в таблице 3 и на рисунке 6.

Таблица 3 – Оценка моделей, обученных на разных выборках

Модель	Accuracy	Precision	Recall	F1_score	Выборка
M1	0.683	0.738	0.683	0.694	SDGi Corpus
M1	0.651	0.629	0.582	0.578	OSDG
M1	0.874	0.891	0.874	0.878	SDGi Corpus+OSDG
M2	0.700	0.746	0.700	0.705	SDGi Corpus
M2	0.680	0.726	0.680	0.685	OSDG
M2	0.881	0.892	0.881	0.884	SDGi Corpus+OSDG
M3	0.848	0.859	0.848	0.850	SDGi Corpus+OSDG

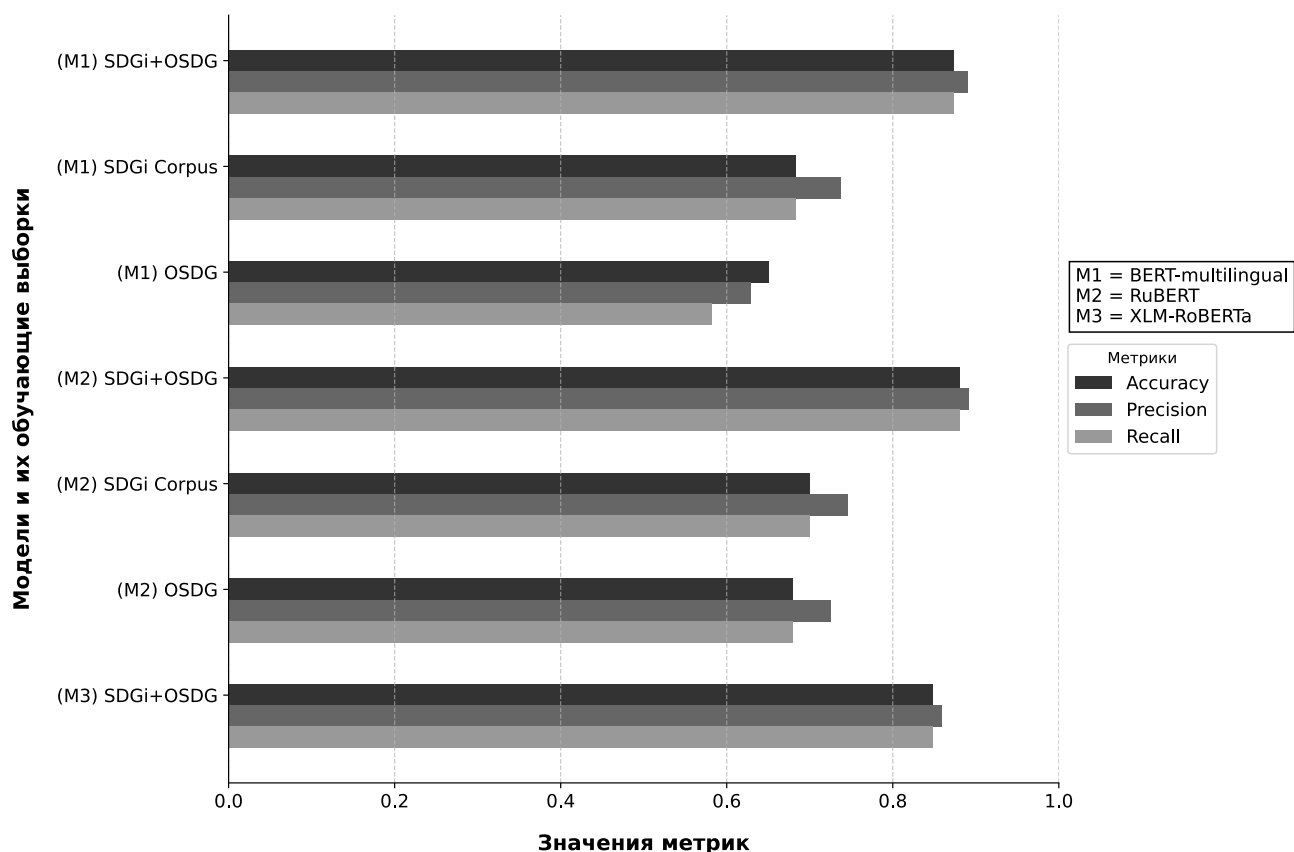


Рисунок 6 – Оценка метрик моделей, дообученных на разных выборках

Все модели обучались 6 эпох, т.к. их метрики на тестовой выборке были максимальны при этом показателе. Обучение происходило на GPU NVIDIA RTX 4060Ti 16 GB, среднее время для одной эпохи:

- 12 минут 43 секунды на объединённой выборке,
- 10 минут 15 секунд на выборке OSDG,
- 2 минут 24 секунды на выборке SDGi Corpus.

Лучшей моделью в эксперименте стала RuBERT (M2), обученная на объединённой выборке SDGi Corpus и OSDG. Она достигла F1-меры 0.884, что превосходит результаты как Multilingual BERT (M1, F1=0.878), так и XLM-RoBERTa (M3, F1=0.850). Это подтверждает преимущество специализированных русскоязычных моделей для обработки локализованных текстов, несмотря на их меньшую языковую универсальность. Рост качества моделей при объединении данных (F1 для M2 вырос с 0.705 до 0.884) демонстрирует критическую важность увеличения объёма обучающих данных для задач классификации ЦУР. Однако достигнутые показатели остаются ниже результатов, полученных для англоязычных аналогов (например, BERT-base в англоязычных работах демонстрирует F1 > 0.90).

Основное ограничение связано с дефицитом экспертно размеченных русскоязычных данных. В англоязычных исследованиях используются корпуса в десятки тысяч документов, аннотированных при участии экспертов ООН. Для русского языка аналогичные ресурсы отсутствуют, что создаёт существенный предел для метрик.

Решением проблемы является формирование специализированного датасета (10–50 тыс. документов) с привлечением экспертов по устойчивому развитию для его разметки.

Текущие результаты подтверждают жизнеспособность подхода, но для достижения высокого качества требуются значительные инвестиции в создание русскоязычных экспертных данных.

3 Реализация интерфейса для классификации текстов

3.1 Разработка приложения для применения BERT-моделей

Для практического применения разработанных моделей создан интерактивный web-интерфейс на базе фреймворка Flask [30], обеспечивающий удобный доступ к функциям классификации. Система позволяет загружать CSV-файлы с текстами, выбирать предобученные модели, выполнять классификацию и скачать размеченные CSV-файлы через графический интерфейс.

Функциональность интерфейса включает несколько ключевых этапов. На этапе загрузки данных пользователь может предоставить CSV-файл с произвольной структурой, после чего система автоматически определяет доступные столбцы, включая текстовые данные. Для предотвращения ошибок реализована валидация формата файла и проверка корректности кодировки. На этапе выбора модели интерфейс динамически загружает доступные чекпоинты из указанной директории, определяя архитектуру (BERT, RuBERT или XLM-R).

Архитектурные особенности решения обеспечивают эффективную обработку данных. Для работы с большими файлами используется пакетная обработка текстов (батчи по 50 элементов), что снижает нагрузку на память GPU. Токенизация выполняется адаптивно: для BERT применяется WordPiece, учитывающая морфологию русского языка, а для XLM-RoBERTa — SentencePiece, оптимизированная для многоязычных контекстов. TensorFlow позволяет задействовать GPU-ускорение, а механизм кэширования промежуточных результатов (например, переводов и токенизаций) сокращает время повторных запросов.

Архитектурные особенности решения обеспечивают эффективную обработку данных. Для работы с большими файлами используется пакетная обработка текстов (батчи по 50 элементов), что снижает нагрузку на память GPU. Токенизация выполняется адаптивно: для BERT применяется токенизатор WordPiece, учитывающий морфологию русского языка, для XLM-RoBERTa — SentencePiece, оптимизированная для многоязычных и сложных контекстов, для RuBERT также используется WordPiece, специально обученный на корпусе русскоязычных текстов.

Визуализация результатов реализована через интерактивную таблицу, где для каждой строки исходного файла отображается предсказанная метка ЦУР, уровень уверенности модели (в процентах). Дополнительно предоставляется возможность экспорта результатов в CSV-формате для последующего анализа

во внешних инструментах.

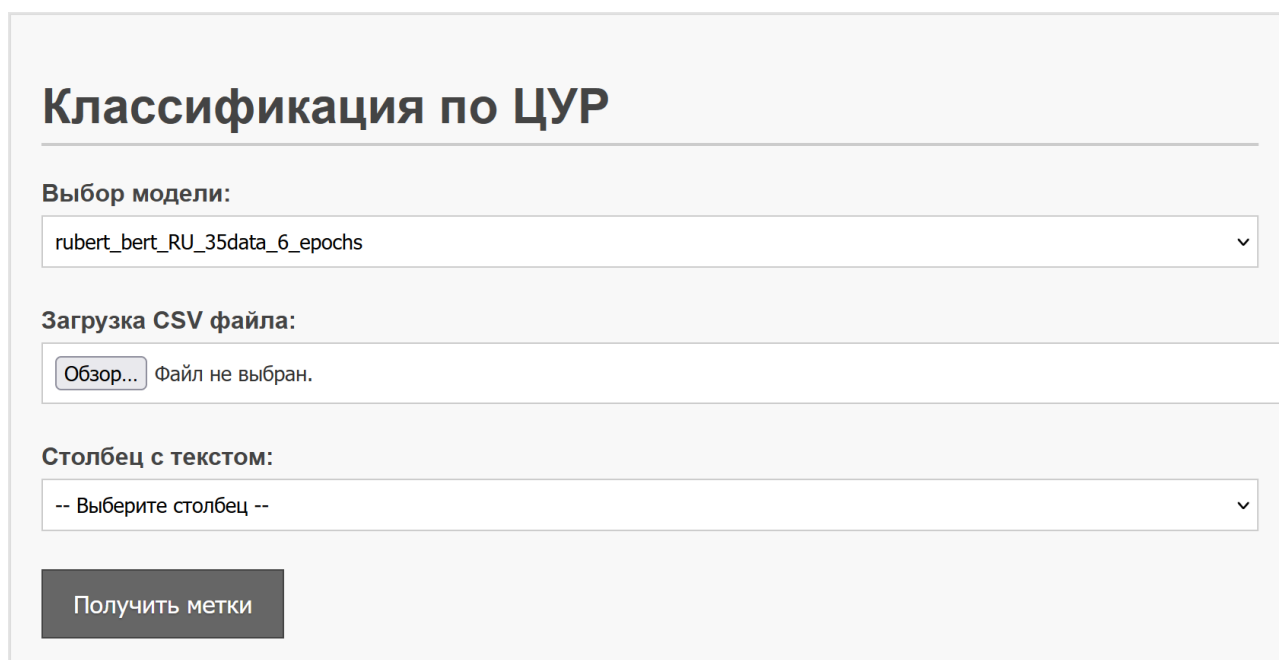
Технологический стек включает Python 3.10, Flask 2.3 для бэкенда, TensorFlow 2.12 для инференса моделей и библиотеку Transformers 4.30 для работы с токенизаторами.

Интерфейс разворачивается как самостоятельное приложение и может быть интегрирован в облачную инфраструктуру. Полный код реализации, включая шаблоны HTML и CSS-стили, доступен на платформе github по ссылке: <https://github.com/StickySmash/bertSDGClassifierInterface>.

3.2 Пример применения приложения для классификации новостей

В качестве набора для примера применения был использован текстовый набор данных, собранный из новостных сообществ различных регионов Российской Федерации. Объём выборки составляет 699 неразмеченных текстов.

Рабочий процесс приложения выглядит следующим образом. Пользователь начинает с загрузки CSV-файла с новостными статьями, который должен содержать столбец с текстом. После загрузки система автоматически определяет доступные столбцы и предлагает пользователю выбрать столбец с текстом в качестве входных данных, что проиллюстрировано на рисунке 7.



Классификация по ЦУР

Выбор модели:

rubert_bert_RU_35data_6_epochs

Загрузка CSV файла:

Обзор... Файл не выбран.

Столбец с текстом:

-- Выберите столбец --

Получить метки

Рисунок 7 – Загрузка датасета и выбор модели в web-интерфейсе

Далее пользователь выбирает модель для анализа и нажимает кнопку Получить метки. После этого модель обрабатывает данные и предсказывает со-

ответствующие метки для каждой статьи. Результаты классификации отображаются в виде таблицы, где для каждой новостной статьи указывается номер соответствующей Цели устойчивого развития (от 1 до 17) и уровень уверенности модели в процентах (например, «SDG 13: 98.68%»).

Завершающим этапом работы с приложением является возможность экспорта размеченного датасета. Интерфейс с результатами классификации и кнопкой экспорта показан на рисунке 8.

Скачать в CSV		
Результаты (Модель: rubert_bert_RU_35data_6_epochs)		
Строка	Метка	Уверенность
1	1	51.27%
2	1	97.14%
3	3	67.36%
4	2	98.93%
5	3	55.04%
6	2	93.37%
7	3	93.38%
8	2	78.04%
9	2	83.38%
10	13	98.68%

Рисунок 8 – Результаты классификации для новостных сообщений

Таким образом, приложение предоставляет удобный и наглядный инструмент для классификации новостных текстов согласно Целям устойчивого развития с возможностью последующего экспорта результатов для дальнейшего анализа. В результате применения модели RuBERT(M2), обученной на совмещённой выборке, были получены метки в соответствии с ЦУР. На рисунке 9 представлен результат классификации.

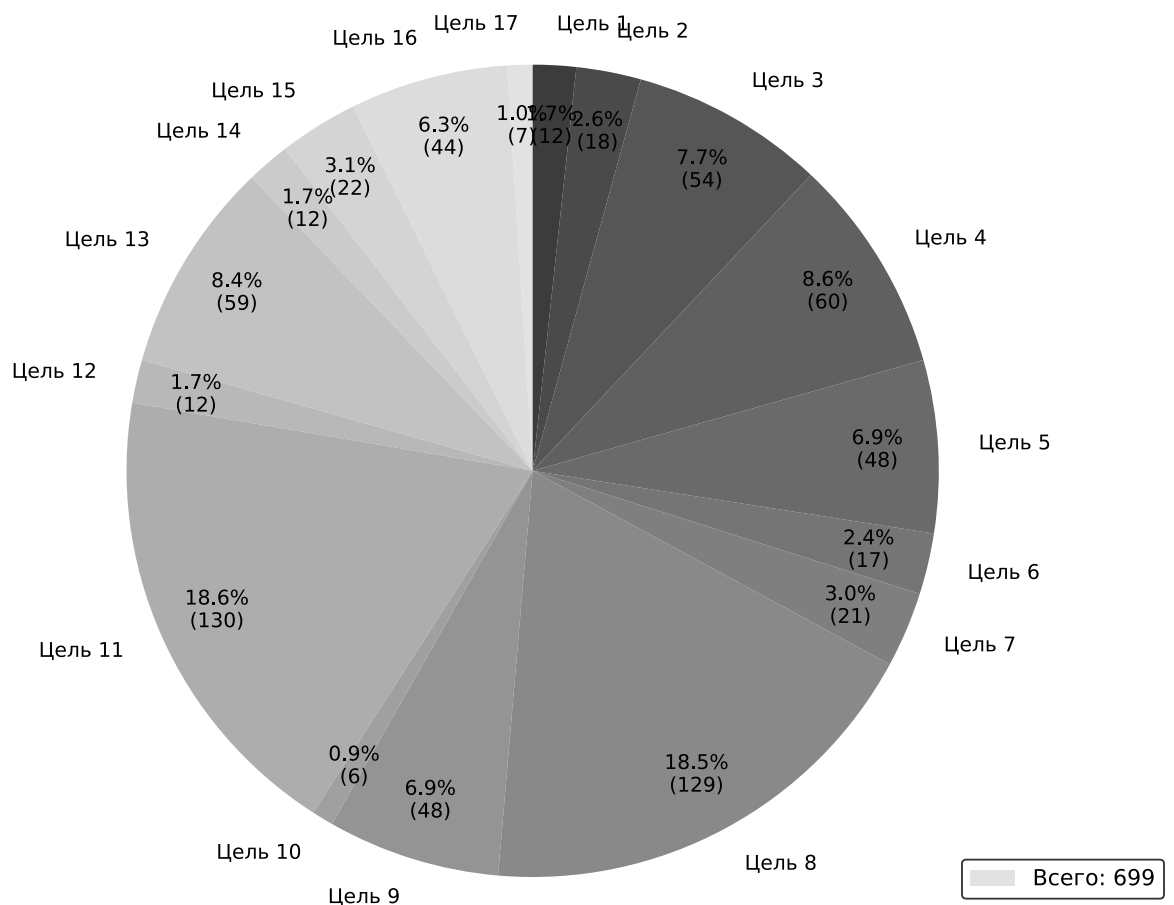


Рисунок 9 – Распределение по ЦУР новостных сообщений

Проведённая классификация позволяет сделать вывод, что тексты связанные с целями 8 и 11 наиболее часто встречаются в выбранном наборе данных.

Разработанный web-интерфейс предоставляет удобный инструмент для автоматической классификации текстовых данных по Целям устойчивого развития. Система поддерживает загрузку CSV-файлов, выбор предобученных моделей (включая BERT, RuBERT и XLM-R), пакетную обработку больших объемов данных и интерактивную визуализацию результатов. Интеграция GPU-ускорения и адаптивных методов токенизации обеспечивает высокую производительность, а возможность экспорта размеченных данных упрощает их дальнейший анализ. Решение демонстрирует эффективность сочетания современных NLP-моделей с пользовательским интерфейсом, что делает его практичным инструментом для исследователей и аналитиков.

ЗАКЛЮЧЕНИЕ

В рамках выполнения бакалаврской работы был разработано приложение для классификации новостных статей по тематике целей устойчивого развития с использованием BERT-моделей.

Были отобраны и адаптированы англоязычные датасеты (OSDG-CD, SDGi Corpus), переведённые на русский язык. На их основе построены модели Multilingual BERT, RuBERT и XLM-RoBERTa для классификации текстов по 17 целям устойчивого развития. Объединение выборок позволило увеличить точность модели для русскоязычных текстов. Наилучший результат показала модель RuBERT ($F1 = 0.884$), что подтверждает преимущество специализированных русскоязычных архитектур. Для практического применения разработан web-интерфейс, обеспечивающий пакетную обработку данных, выбор моделей и экспорт результатов, что делает решение удобным инструментом для аналитиков.

Ключевым результатом работы стало подтверждение гипотезы о применимости дообучения для задач классификации ЦУР, даже при ограниченном объёме русскоязычных данных. Однако выявлены ограничения:

- обучение BERT-моделей требует значительных вычислительных мощностей и эффективной параллелизации процессов, особенно при работе с большими объёмами данных;
- дефицит экспертно размеченных русскоязычных датасетов, снижающий точность моделей по сравнению с англоязычными аналогами;
- дисбаланс классов в обучающих выборках, который требует дополнительной аугментации и взвешивания меток.

Перспективными направлениями для будущих исследований являются:

- создание открытого русскоязычного корпуса текстов, аннотированных экспертами в области ЦУР;
- разработка мультимодальных моделей, учитывающих не только текст, но и структурные элементы документов (графики, таблицы);
- интеграция методов активного обучения для сокращения затрат на разметку данных.

Практическая значимость работы заключается в автоматизации анализа медиатекстов, что ускоряет мониторинг прогресса в достижении ЦУР и поддержку принятия решений в государственном и корпоративном секторах. Реализованное решение демонстрирует потенциал BERT-архитектур для обработ-

ки русскоязычных данных и может служить основой для дальнейшего развития инструментов NLP в контексте устойчивого развития.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. (2019). Text Classification Algorithms: A Survey. *Information*, 10(4), 150. <https://doi.org/10.3390/info10040150>
- 2 Kotsiantis, S. B., Zaharakis, I. D., Pintelas, P. E. Machine learning: a review of classification and combining techniques // *Artificial Intelligence Review*. – 2006. – Т. 26. – С. 159-190.
- 3 Gasparetto, A., Marcuzzo, M., Zangari, A., & Albarelli, A. (2022). A Survey on Text Classification Algorithms: From Text to Predictions. *Information*, 13(2), 83. <https://doi.org/10.3390/info13020083>
- 4 Devlin, J., et al. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- 5 Vaswani, A. et al. Attention is all you need // *Advances in neural information processing systems*. – 2017. – Т. 30.
- 6 Ferrando, J. et al. A primer on the inner workings of transformer-based language models // *arXiv preprint arXiv:2405.00208*. – 2024.
- 7 Tida, V. S., Hsu, S. Universal spam detection using transfer learning of BERT model // *arXiv preprint arXiv:2202.03480*. – 2022.
- 8 Understanding Activation Functions in Neural Networks // *Medium.com*
URL: <https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0> (дата обращения: 27.04.2025).
- 9 Klebanov, L., Rachev, S.T., Fabozzi, F. *Robust and Non-Robust Models in Statistics*, Nova Scientific Publishers, New York, 2009.
- 10 Lin, T. Y. et al. Focal loss for dense object detection // *Proceedings of the IEEE international conference on computer vision*. – 2017. – С. 2980-2988.
- 11 Okewu, E., Adewole, P., Sennaike, O. Experimental comparison of stochastic optimizers in deep learning // *Computational Science and Its Applications–ICCSA 2019: 19th International Conference, Saint Petersburg, Russia, July 1–4, 2019, Proceedings, Part V 19*. – Springer International Publishing, 2019. – С. 704-715.

- 12 Оценка качества в задачах классификации // neerc.ifmo.ru URL: https://neerc.ifmo.ru/wiki/index.php?title=Оценка_качества_в_задачах_классификации (дата обращения: 28.04.2025).
- 13 Цели устойчивого развития: Повестка дня на период до 2030 года // ООН. URL: <https://www.un.org/sustainabledevelopment/ru> (дата обращения: 15.05.2025).
- 14 Global Indicator Framework for the Sustainable Development Goals // UN Statistics Division. URL: <https://unstats.un.org/sdgs/indicators/indicators-list> (дата обращения: 15.05.2025).
- 15 Селиванова, И.В. Классификация научных исследований целей устойчивого развития ООН: проблемы, подходы и перспективы использования генеративного искусственного интеллекта / И. В. Селиванова, П. Ю. Блинов, А. В. Малышева, Д. В. Косяков // Научные и технические библиотеки. – 2025. – № 1. – С. 56–78. – DOI: 10.33186/1027-3689-2025-1-56-78.
- 16 Digital News Report 2023 // Reuters Institute. URL: <https://reutersinstitute.politics.ox.ac.uk/digital-news-report/2023> (дата обращения: 15.05.2025).
- 17 Guisiano, J. Automatic classification of multilabel texts related to Sustainable Development Goals (SDGs) / J. Guisiano, R. Chiky // Proceedings of the TECHENV EGC2021. – Montpellier, France, Jan. 2021. – URL: <https://hal.science/hal-03154261v1> (дата обращения: 16.04.2025).
- 18 Angin, M. A RoBERTa Approach for Automated Processing of Sustainability Reports / M. Angin, B. Tasdemir, C. A. Yilmaz, G. Demiralp, M. Atay, P. Angin, G. Dikmener // Sustainability . – 2022. – Vol. 14, № 23. – P. 16139. – DOI: 10.3390/su142316139 .
- 19 Казакова, М. А., Султанова, А. П. Анализ технологии обработки естественного языка: современные проблемы и подходы //Advanced Engineering Research (Rostov-on-Don). – 2022. – Т. 22. – №. 2. – С. 169-176.
- 20 EUR-Lex SDG-Annotated Dataset // Hugging Face URL: https://huggingface.co/datasets/razaulhaq/eurlex_sdg_coverage (дата обращения: 14.04.2025).

- 21 UNO Sustainable Development Goals // Hugging Face URL: https://huggingface.co/datasets/Rodion/uno_sustainable_development_goals (дата обращения: 14.04.2025).
- 22 OSDG Community Dataset (OSDG-CD) // Zenodo URL: <https://zenodo.org/records/11441197> (дата обращения: 13.04.2025).
- 23 SDG-single // Hugging Face URL: <https://huggingface.co/datasets/dspoka/sdg-single> (дата обращения: 14.04.2025).
- 24 SDGi Corpus // Hugging Face URL: <https://huggingface.co/datasets/UNDP/sdgi-corpus> (дата обращения: 14.04.2025).
- 25 OSDG SDG Dataset // Hugging Face URL: https://huggingface.co/datasets/jonas/osdg_sdg_dataset (дата обращения: 14.04.2025).
- 26 SDGdescriptions-AURORA05v2 // Hugging Face URL: https://huggingface.co/datasets/gorkaartola/ZS-train_S1-SDGdescriptions-AURORA05v2_S2-SDGdescriptions-SDGtitle_NSF-SDGdesc-AURORA05v2 (дата обращения: 14.04.2025).
- 27 BERT multilingual base model (cased) // Kaggle URL: <https://www.kaggle.com/datasets/aditidutta/bert-base-multilingual-cased> (дата обращения: 16.04.2025).
- 28 RuBERT // Hugging Face URL: <https://huggingface.co/DeepPavlov/rubert-base-cased> (дата обращения: 16.04.2025).
- 29 XLM-RoBERTa (base-sized model) // Hugging Face URL: <https://huggingface.co/FacebookAI/xlm-roberta-base> (дата обращения: 16.04.2025).
- 30 Flask // Flask URL: <https://flask.palletsprojects.com/en/stable/> (дата обращения: 08.05.2025).

ПРИЛОЖЕНИЕ А

Программный код для предобработки набора данных

```
import pandas as pd
import numpy as np
import os
import nltk
import pymorphy2
from tqdm import tqdm

russian_stopwords = set(nltk.corpus.stopwords.words('russian')) | {
    'sdg', 'csdg', 'xiv', 'hlpf', 'цур', 'год', 'также', 'который',
    'цель', 'развитие', 'национальный', 'устойчивый', 'человек',
    'уровень', 'программа', 'страна', 'процент', 'оон'
}

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[\^w\s]|\d', ' ', text) # Удаляем пунктуацию и цифры
    text = re.sub(r'\s+', ' ', text).strip() # Удаляем лишние пробелы

    # Фильтрация стоп-слов
    words = [word for word in text.split()
              if len(word) > 2 and word not in russian_stopwords]

    return ' '.join(words)

ds['text_clean'] = ds['text'].apply(clean_text)

morph = pymorphy2.MorphAnalyzer()

# Функция для лемматизации одной строки
def lemmatize_text(text):
    words = text.split()
    lemmatized_words = [morph.parse(word)[0].normal_form for word in words]
    return ' '.join(lemmatized_words)

# Включение tqdm для pandas
tqdm.pandas(desc="Лемматизация всего датасета")
ds8['cleaned_lemmatized'] = ds8['cleaned'].progress_apply(lemmatize_text)
```

ПРИЛОЖЕНИЕ Б

Программный код для перевода текстов

```
import os
import re
import time
import random
import pandas as pd
import nltk
from tqdm import tqdm
from deep_translator import GoogleTranslator, exceptions
from concurrent.futures import ThreadPoolExecutor, as_completed
from functools import partial

# =====
# НАСТРОЙКИ ПУТЕЙ
# =====
BASE_DIR = "C:\\Users\\stick\\OneDrive\\Документы\\GitHub\\diplom\\code\\
    datasets\\translate\\8DATA"
OUTPUT_CSV = os.path.join(BASE_DIR, "final_processed.csv")
CACHE_DIR = os.path.join(BASE_DIR, "cache")
CACHE_FILE = os.path.join(CACHE_DIR, "translation_cache.csv")
BATCH_SAVE_DIR = os.path.join(BASE_DIR, "processed_batches")

# Создаем необходимые директории
os.makedirs(CACHE_DIR, exist_ok=True)
os.makedirs(BATCH_SAVE_DIR, exist_ok=True)

# =====
# ПАРАМЕТРЫ ОБРАБОТКИ
# =====
MAX_ATTEMPTS = 3                # Максимальное количество попыток перевода
MAX_TEXT_LENGTH = 4000          # Максимальная длина текста для одного запроса
CACHE_SAVE_INTERVAL = 1000      # Сохранять кэш каждые 100 переводов
BASE_DELAY = 1.5                # Базовая задержка между запросами
JITTER = 0.5                    # Случайная добавка к задержке
NUM_WORKERS = 6                 # Количество потоков для параллельной
                                # обработки
BATCH_SIZE = 50                 # Размер пакета для сохранения результатов

# =====
# ИНИЦИАЛИЗАЦИЯ NLTK
```

```

# =====
nltk.download('stopwords', quiet=True)
russian_stopwords = set(nltk.corpus.stopwords.words('russian')) | {'это', '
    весь'}

# =====
# ЗАГРУЗКА КЭША
# =====
translation_cache = {}
if os.path.exists(CACHE_FILE):
    try:
        cache_df = pd.read_csv(CACHE_FILE)
        translation_cache = dict(zip(cache_df['original'], cache_df['
            translated']))
        print(f"Загружен кэш переводов: {len(translation_cache)} записей")
    except Exception as e:
        print(f"Ошибка загрузки кэша: {e}. Начинаем с пустого кэша")

# =====
# ФУНКЦИИ ОБРАБОТКИ
# =====
def save_cache():
    """Сохраняет текущий кэш переводов в файл"""
    try:
        cache_df = pd.DataFrame(list(translation_cache.items()), columns=['
            original', 'translated'])
        cache_df.to_csv(CACHE_FILE, index=False)
        print(f"\nКэш сохранен: {len(cache_df)} записей")
    except Exception as e:
        print(f"\nОшибка сохранения кэша: {e}")

def translate_text(text):
    """Улучшенная функция перевода с обработкой ошибок и разделением текста"""
    if pd.isna(text) or not (text := str(text).strip()):
        return ""

    if text in translation_cache:
        return translation_cache[text]

    # Разделяем длинные тексты на части
    text_parts = [text[i:i+MAX_TEXT_LENGTH] for i in range(0, len(text),

```

```

        MAX_TEXT_LENGTH)]
translated_parts = []

for part in text_parts:
    attempt = 0
    while attempt < MAX_ATTEMPTS:
        try:
            translated = GoogleTranslator(source='en', target='ru').
            translate(part)
            time.sleep(2)
            translated_parts.append(translated)
            break
        except exceptions.TranslationNotFound as e:
            print(f"\nОшибка перевода: {str(e)}")
            time.sleep(10 * (attempt + 1))
            attempt += 1
        except Exception as e:
            print(f"\nКритическая ошибка: {str(e)}")
            return ""

full_translation = ' '.join(translated_parts)
translation_cache[text] = full_translation

if len(translation_cache) % CACHE_SAVE_INTERVAL == 0:
    save_cache()

return full_translation

def clean_text(text):
    """Улучшенная очистка текста"""
    text = re.sub(r'[^w\s]|\d', ' ', str(text).lower())
    text = re.sub(r'\s+', ' ', text).strip()
    return ' '.join(word for word in text.split()
                      if len(word) > 2 and word not in russian_stopwords)

def process_single_row(row):
    """Обработка одной строки данных"""
    try:
        translated = translate_text(row['text'])
        cleaned = clean_text(translated)
        return {

```

```

        **row.to_dict(),
        'translated': translated,
        'cleaned': cleaned
    }
except Exception as e:
    print(f"Ошибка обработки строки {row.name}: {e}")
    return None

# =====
# ОСНОВНАЯ ЛОГИКА
# =====
def main():
    print("Загрузка данных...")
    df = clean_data_lower_300_and_1label

    print(f"Начало обработки {len(df)} записей...")
    start_time = time.time()

    results = []
    processed_count = 0

    with ThreadPoolExecutor(max_workers=NUM_WORKERS) as executor:
        futures = [executor.submit(process_single_row, row) for _, row in df.
                    iterrows()]

        for future in tqdm(as_completed(futures), total=len(futures), desc="
Обработка"):
            result = future.result()
            if result:
                results.append(result)
                processed_count += 1

        # Сохранение промежуточных результатов
        if processed_count % BATCH_SIZE == 0:
            batch_df = pd.DataFrame(results[-BATCH_SIZE:])
            batch_file = os.path.join(BATCH_SAVE_DIR, f"batch_{
processed_count}.csv")
            batch_df.to_csv(batch_file, index=False)
            time.sleep(5)

```

```
# Финализация
save_cache()
final_df = pd.DataFrame(results)
final_df.to_csv(OUTPUT_CSV, index=False)

elapsed = time.time() - start_time
print(f"\nОбработка завершена за {elapsed:.2f} секунд")
print(f"Сохранено записей: {len(final_df)}")
print(f"Финальный файл: {OUTPUT_CSV}")

if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ В

Программный код для обучения моделей трансформеров

#модель Multilingual

```
def multi_model(num_classes):
    model_name = "bert-base-multilingual-cased"
    # Загрузка предобученной модели
    base_model = CustomMultilingualBert.from_pretrained(
        model_name,
        num_labels=num_classes,
        # problem_type="multi_label_classification"
    )

    # Входные слои
    input_ids = layers.Input(shape=(None,), dtype=tf.int32, name="input_ids")
    attention_mask = layers.Input(shape=(None,), dtype=tf.int32, name="
        attention_mask")

    # Выход базовой модели
    bert_outputs = base_model(input_ids=input_ids, attention_mask=
        attention_mask).logits
    cls_output = bert_outputs
    # Дополнительная обработка для учета взаимосвязей меток
    x = layers.Dense(256, activation="relu")(cls_output) # Слой для выявления
        связей
    x = tf.keras.layers.Dropout(0.3)(x) # Для борьбы с переобучением
    # Финальный слой
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    # Сборка модели
    model = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=outputs
        )

    # Метрики с микро-усреднением
    metrics = [
        tf.keras.metrics.CategoricalAccuracy(
            name="accuracy",
        ),
        tf.keras.metrics.Precision(
            name="precision", top_k=1
        ), # для категориальных данных
```

```

        tf.keras.metrics.Recall(name="recall", top_k=1),
        tfa.metrics.F1Score(num_classes=num_classes, average="macro", name="f1
    ),
]

```

Компиляция

```

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    # loss='categorical_crossentropy',
    loss=tf.keras.losses.CategoricalCrossentropy(),
    metrics=metrics,
)

```

return model

модель RuBERT

```

def rubert_model(num_classes):
    # Загрузка предобученной модели
    base_model = TFAutoModelForSequenceClassification.from_pretrained(
        "DeepPavlov/rubert-base-cased", num_labels=num_classes, from_pt=True
    )

    # Входные слои
    input_ids = layers.Input(shape=(None,), dtype=tf.int32, name="input_ids")
    attention_mask = layers.Input(shape=(None,), dtype=tf.int32, name="
        attention_mask")

    # Выход базовой модели
    bert_outputs = base_model(input_ids=input_ids, attention_mask=
        attention_mask).logits
    cls_output = bert_outputs
    # Дополнительная обработка для учета взаимосвязей меток
    x = layers.Dense(256, activation="relu")(cls_output) # Слой для выявления
        связей
    x = tf.keras.layers.Dropout(0.3)(x) # Для борьбы с переобучением
    # Финальный слой
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    # Сборка модели

```



```

model = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=outputs
    )

# Метрики с микро-усреднением
metrics = [
    tf.keras.metrics.CategoricalAccuracy(
        name="accuracy",
    ),
    tf.keras.metrics.Precision(
        name="precision", top_k=1
    ), # для категориальных данных
    tf.keras.metrics.Recall(name="recall", top_k=1),
    tfa.metrics.F1Score(num_classes=num_classes, average="macro", name="f1
    "),
]

# Компиляция
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    # loss='categorical_crossentropy',
    loss=tf.keras.losses.CategoricalCrossentropy(),
    metrics=metrics,
)

return model

#модель xlmr
def xlmr_model(num_classes):
    # Загрузка предобученной модели
    base_model = CustomXLMRoberta.from_pretrained(
        "xlm-roberta-base",
        num_labels=num_classes,
        # problem_type="multi_label_classification"
    )

    # Входные слои
    input_ids = layers.Input(shape=(None,), dtype=tf.int32, name="input_ids")
    attention_mask = layers.Input(shape=(None,), dtype=tf.int32, name="
        attention_mask")

```

```

# Выход базовой модели
bert_outputs = base_model(input_ids=input_ids, attention_mask=
    attention_mask).logits
cls_output = bert_outputs
# Дополнительная обработка для учета взаимосвязей меток
x = layers.Dense(256, activation="relu")(cls_output) # Слой для выявления
    связей
x = tf.keras.layers.Dropout(0.3)(x) # Для борьбы с переобучением
# Финальный слой
outputs = layers.Dense(num_classes, activation="softmax")(x)

# Сборка модели
model = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=outputs
    )

# Метрики с микро-усреднением
metrics = [
    tf.keras.metrics.CategoricalAccuracy(name="accuracy"),
    tf.keras.metrics.Precision(
        name="precision", top_k=1
    ), # для категориальных данных
    tf.keras.metrics.Recall(name="recall", top_k=1),
    tfa.metrics.F1Score(
        num_classes=num_classes,
        average="weighted", # или 'micro', если классы сбалансированы
    ),
]

# Компиляция
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    loss="categorical_crossentropy",
    metrics=metrics,
)

return model

```

Бакалаврская работа "Разработка приложения для классификации новостных статей с помощью BERT-моделей" выполнена мною самостоятельно, и на все источники, имеющиеся в работе, даны соответствующие ссылки.

подпись, дата

инициалы, фамилия